LeMi
Fire Spread

CONTENTS

1. Introduction

All of my assets are created based on the game I'm developing.
While building tools to speed up game production, I also organized some useful resources to share with fellow creators.

This asset includes features below:

- **Fire Shader**: A highly customizable shader for creating detailed fire effects.
- **Burn Map Propagation Algorithm**: Simulates multiple fire stages, including burning, waiting, and recovery.
- **Terrain Alignment**: Ensures particle effects spawn accurately according to terrain height.
- **GPU Acceleration**: Utilizes Compute Shaders for high-performance fire spread simulation.
- **Collision Detection**: Supports trigger events when objects enter or exit burning areas.

---

2. Fire Spread

LeMi – Fire Spread is a GPU-accelerated fire propagation system designed for Unity's Universal Render Pipeline (URP).

It uses Compute Shaders for high-performance simulation and integrates with VFX Graph to deliver realistic, dynamic fire visuals.

⚠️ The grass interaction effect is **not included** in this asset.

You can implement your own system or use my separate package

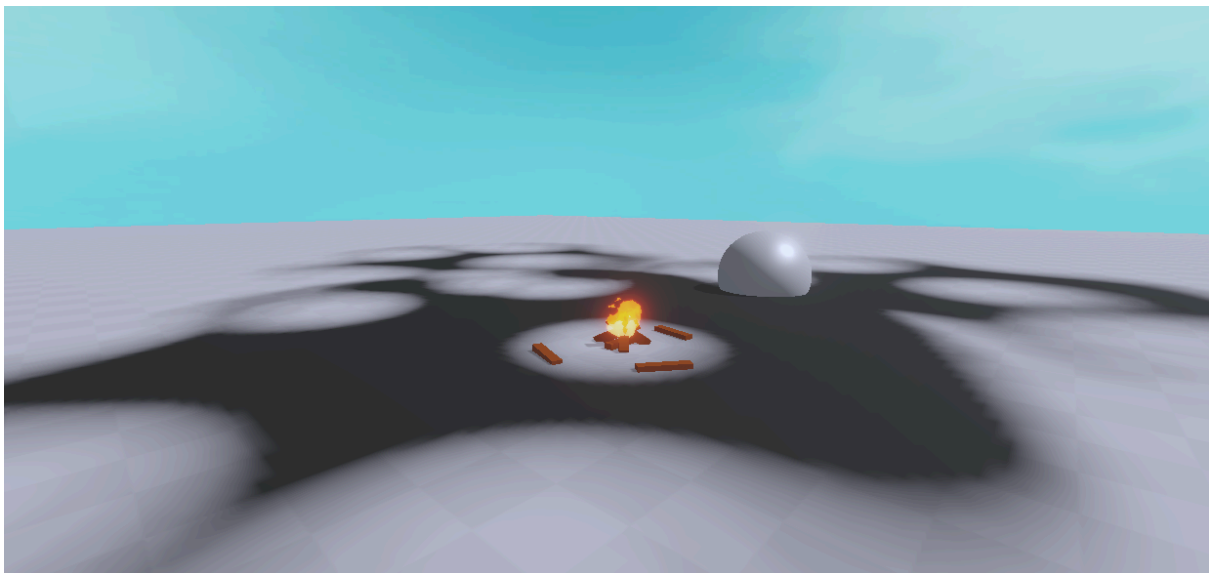
LeMi – Interactive Grass & Detail Generator

Both systems are designed to function independently, as not all games require fire simulation.

This package is built specifically to complement my grass system

---

Setup Guide



Step 1: Prepare the Scene

1. Create a new scene or open an existing one.
2. Make sure a Terrain object exists in the scene.
3. Confirm your project is using the Universal Render Pipeline (URP).

Step 2: Create a Burn Map

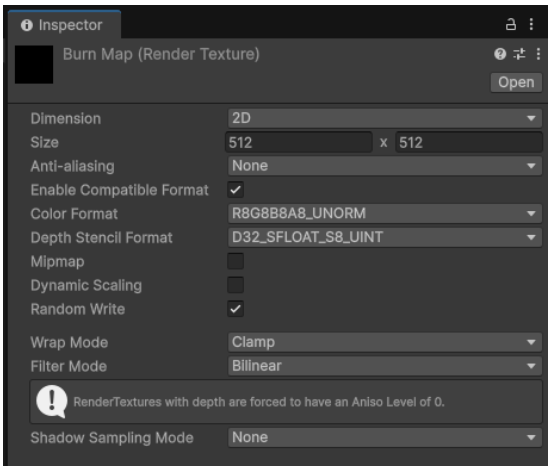
1. In the Project window, right-click and select
   Create / Rendering / Render Texture.

2. Name it BurnMap.

3. Set the resolution (512×512 or higher).

4. Change the Format to ARGB 32-bit.

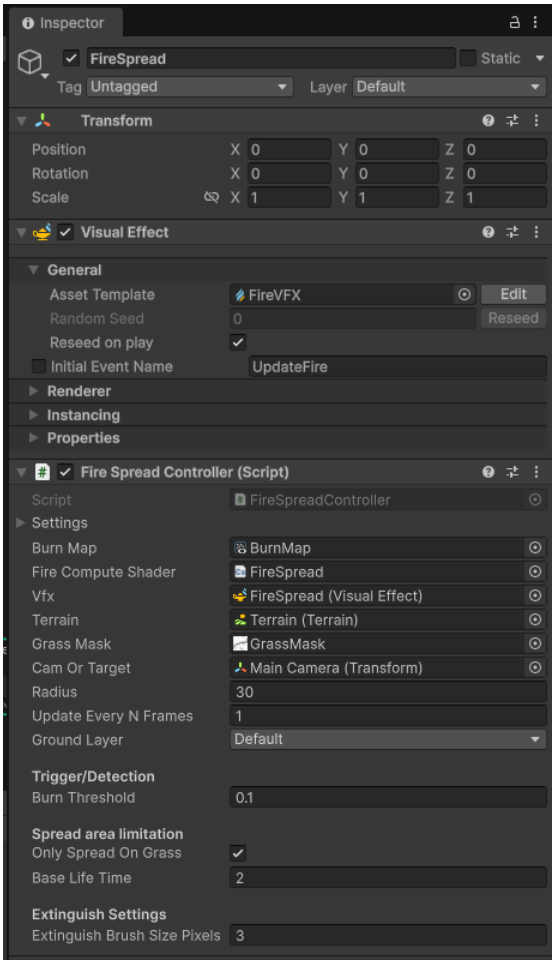
5. Enable Random Write.



Step 3: Configure the FireSpreadController

1. In the Hierarchy, create an empty GameObject and name it FireSpreadController.

2. Add the FireSpreadController component.
3. Add the Visual Effect component and assign FireVFX (Runtime/VFX/FireVFX.vfx)
4. In the Inspector, assign the following properties:



Required Settings

- Burn Map – Assign the Render Texture you created earlier.

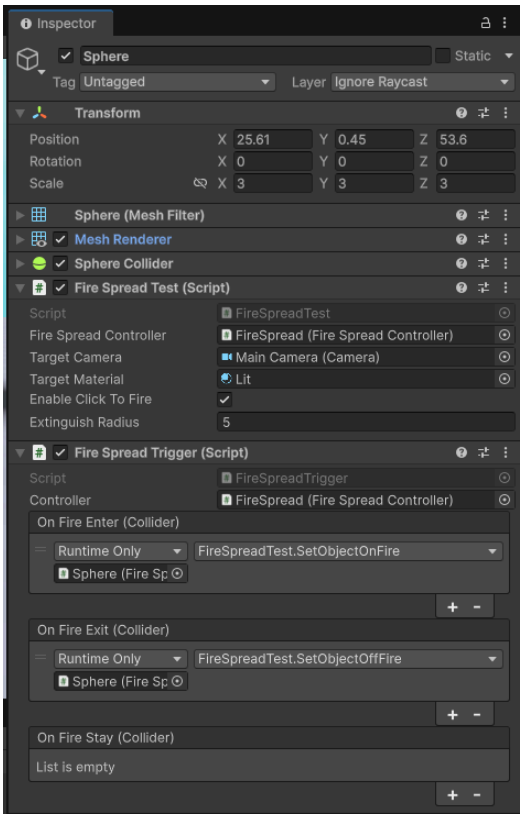
- Fire Compute Shader – Drag in FireSpread.compute (found under

Runtime/Shaders/).

- VFX – Assign the fire VFX prefab (Runtime/VFX/FireVFX.vfx).
- Terrain – Drag in your Terrain object (leave empty to auto-detect).

Optional Settings

- Grass Mask – A texture mask that limits fire propagation to specific areas.
- Cam Or Target – A camera or target transform for visibility culling.

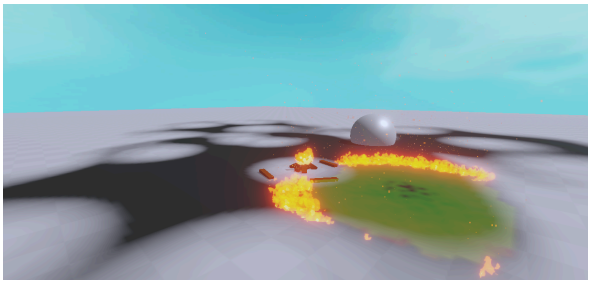


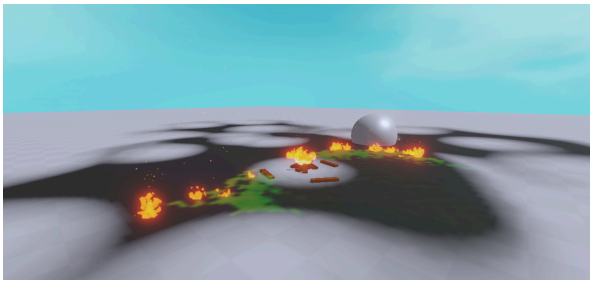
Step 4: Test the Fire Simulation

1. Create a small test script or use the provided FireSpreadTest.cs
2. Assign fields and click play

3. Run the scene

1. right click on screen to start fire
2. left click to extinguish fire



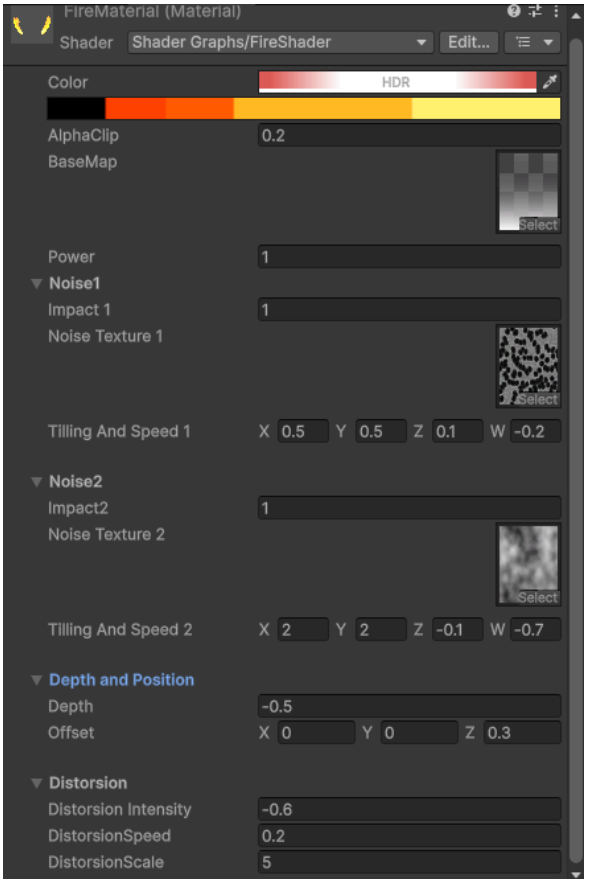


If you enable Only Spread On Grass option, it will only burning on valid grass mask position

3. Fire Shader

This asset includes a **highly customizable Fire Shader** designed for flexible use and visual variety.

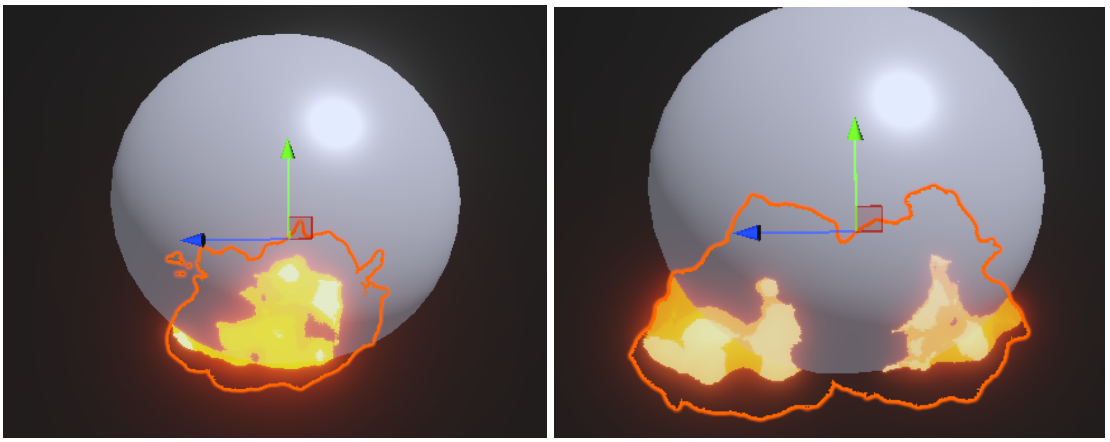
The core function is simple: the **BaseMap** defines the main transition area, while multiple moving **Noise layers** are blended together to simulate the dynamic feel of burning flames.
By adjusting different settings and textures, you can easily create unique fire styles — though, be warned, tweaking shaders can be surprisingly addictive!



- **Power** controls the overall intensity and blend strength.
- **Distance** adjusts the rendering depth of the flames.
- **Texture**, **Impact**, and **Noise Tiling** (each with two sets) fine-tune the layered noise effects for diverse flame patterns.
- **Distortion** is used to create a disturbance effect on the outer layer of a flame.

In this asset, the shader is used purely for **visual effects** and is **not directly connected** to the fire propagation logic. You can freely replace the fire material with your own at any time.

As for **Distance** and **Offset**, they are used to give the flames a sense of **volume and curvature**.



Offset subtly shifts the fire toward the player's viewpoint, enhancing the illusion of thickness, while Distance adjusts the flame's curvature—creating a slightly rounded appearance that wraps around objects when viewed from the side.